

Homework 7: Variational inference

Bernhard Buss Dario Franzen Aurimas Greicius
Timothy Lauren Nijhuis Carlo Rossetto Yuliyang Slavchev
Görkem Kadir Solun

```
set.seed(42)
```

Problem 17: Variational inference

Part a: Write down the full joint probability & log-probability of the data and latent variables.

Let $x = (x_1, \dots, x_N)$, $z = (z_1, \dots, z_N)$ with $z_i \in \{1, \dots, K\}$, mixing proportions $\pi = (\pi_1, \dots, \pi_K)$, and component parameters $p = (p_1, \dots, p_K)$. The model is:

$$p(x_i \mid z_i = k, p_k) = \binom{x_i + r - 1}{x_i} p_k^r (1 - p_k)^{x_i}, \quad p(z_i = k \mid \pi) = \pi_k,$$

with priors

$$\pi \sim \text{Dirichlet}(\alpha_{0,1}, \dots, \alpha_{0,K}), \quad p_k \sim \text{Beta}(a_0, b_0).$$

Hence the full joint factorization is

$$p(x, z, \pi, p) = p(\pi) \prod_{k=1}^K p(p_k) \prod_{i=1}^N p(z_i \mid \pi) p(x_i \mid z_i, p).$$

Using one-hot indicators $\mathbb{1}[z_i = k]$:

$$p(x, z, \pi, p) = \frac{1}{B(\alpha_0)} \prod_{k=1}^K \pi_k^{\alpha_{0,k}-1} \prod_{k=1}^K \frac{1}{B(a_0, b_0)} p_k^{a_0-1} (1-p_k)^{b_0-1} \prod_{i=1}^N \prod_{k=1}^K \left[\pi_k \binom{x_i + r - 1}{x_i} p_k^r (1 - p_k)^{x_i} \right]^{\mathbb{1}[z_i=k]}.$$

Therefore, the log-joint is

$$\begin{aligned}\log p(x, z, \pi, p) = & -\log B(\alpha_0) + \sum_{k=1}^K (\alpha_{0,k} - 1) \log \pi_k \\ & + \sum_{k=1}^K \left[-\log B(a_0, b_0) + (a_0 - 1) \log p_k + (b_0 - 1) \log(1 - p_k) \right] \\ & + \sum_{i=1}^N \sum_{k=1}^K \mathbb{1}[z_i = k] \left[\log \pi_k + \log \binom{x_i + r - 1}{x_i} + r \log p_k + x_i \log(1 - p_k) \right].\end{aligned}$$

Part b: Derive the variational update for $Q(\mathbf{z})$

Use mean-field:

$$Q(\pi, p, z) = Q(\pi) \prod_{k=1}^K Q(p_k) \prod_{i=1}^N Q(z_i),$$

with $Q(\pi) = \text{Dirichlet}(\alpha)$ and $Q(p_k) = \text{Beta}(a_k, b_k)$.

For $Q(z_i)$:

$$\log Q^*(z_i = k) \propto \mathbb{E}_{Q(\pi)Q(p_k)} [\log \pi_k + r \log p_k + x_i \log(1 - p_k)].$$

Using Dirichlet/Beta expectations,

$$\mathbb{E}[\log \pi_k] = \psi(\alpha_k) - \psi\left(\sum_{j=1}^K \alpha_j\right),$$

$$\mathbb{E}[\log p_k] = \psi(a_k) - \psi(a_k + b_k), \quad \mathbb{E}[\log(1 - p_k)] = \psi(b_k) - \psi(a_k + b_k),$$

so responsibilities are

$$\rho_{ik} := Q(z_i = k) \propto \exp \left\{ \psi(\alpha_k) - \psi\left(\sum_{j=1}^K \alpha_j\right) + r [\psi(a_k) - \psi(a_k + b_k)] + x_i [\psi(b_k) - \psi(a_k + b_k)] \right\},$$

then normalize:

$$Q(z_i = k) = \frac{\rho_{ik}}{\sum_{\ell=1}^K \rho_{i\ell}}, \quad \sum_{k=1}^K Q(z_i = k) = 1.$$

Part c: Derive the variational update for $Q(\cdot)$

Collect terms involving π :

$$\log Q^*(\pi) \propto \sum_{k=1}^K \left(\alpha_{0,k} - 1 + \sum_{i=1}^N \mathbb{E}[\mathbb{1}[z_i = k]] \right) \log \pi_k.$$

Since $\mathbb{E}[\mathbb{1}[z_i = k]] = Q(z_i = k)$, we get

$$Q(\pi) = \text{Dirichlet}(\alpha_1, \dots, \alpha_K),$$

$$\alpha_k = \alpha_{0,k} + \sum_{i=1}^N Q(z_i = k).$$

Part d: Derive the variational update for $Q(p_k)$

Collect terms involving one p_k :

$$\log Q^*(p_k) \propto \left(a_0 - 1 + r \sum_{i=1}^N \mathbb{E}[\mathbb{1}[z_i = k]] \right) \log p_k + \left(b_0 - 1 + \sum_{i=1}^N x_i \mathbb{E}[\mathbb{1}[z_i = k]] \right) \log(1 - p_k).$$

Thus

$$Q(p_k) = \text{Beta}(a_k, b_k),$$

$$a_k = a_0 + r \sum_{i=1}^N Q(z_i = k), \quad b_k = b_0 + \sum_{i=1}^N Q(z_i = k) x_i.$$

These are the CAVI updates used iteratively with the $Q(z_i)$ update until convergence of the ELBO (or parameter change threshold).

Problem 18: R-Implementation of variational inference updating equations for a Negative Binomial mixture of cell populations

Part a: Implement the above derived variational inference procedure to estimate k and σ^2 for the data in `vi.csv`.

We use the parameterization

$$X \mid z_i = k \sim \text{NegBin}(r, p_k), \quad \mathbb{E}[X \mid z_i = k] = \lambda_k = \frac{r(1 - p_k)}{p_k}.$$

```
# Data loading helper
load_vi_counts <- function(preferred_col = NULL) {
  csv_candidates <- c("vi.csv", "hw7/vi.csv")
  csv_path <- csv_candidates[file.exists(csv_candidates)][1]

  if (!is.na(csv_path)) {
    dat <- read.csv(csv_path)
  } else {
    stop("vi.csv not found (checked local folder and hw7/).")
  }

  # Select count column.
  if (!is.null(preferred_col) && preferred_col %in% names(dat)) {
    x <- dat[[preferred_col]]
  } else {
    numeric_cols <- vapply(dat, is.numeric, logical(1))
    if (!any(numeric_cols)) {
      stop("No numeric column found in input file.")
    }

    idx <- which(numeric_cols)
    nms <- names(dat)[idx]

    # Drop index-like columns if possible (typical in exported spreadsheets).
    index_like <- grepl("^((\\.\\.|\\.\\.|[0-9]+|id|index|cell(?:_?id)?)$", tolower(nms))
    if (any(!index_like)) {
      idx <- idx[!index_like]
      nms <- nms[!index_like]
    }

    # Prefer biologically meaningful names when available.
```

```

preferred_name <- grepl("atp|count|expr|expression|gene", tolower(nms))
if (any(preferred_name)) {
  idx <- idx[preferred_name]
}

# Final fallback: choose column with smallest unique-ratio (less likely to be row index)
unique_ratio <- vapply(
  dat[idx],
  function(col) length(unique(col[!is.na(col)])) / sum(!is.na(col)),
  numeric(1)
)
x <- dat[[idx[which.min(unique_ratio)]]]
}

if (any(is.na(x))) {
  x <- x[!is.na(x)]
}

x <- as.integer(round(x))
if (any(x < 0)) {
  stop("Counts must be non-negative.")
}

x
}

# CAVI for NB mixture
run_nb_vi <- function(x,
  K = 2,
  r = 10,
  a0 = 1,
  b0 = 1,
  alpha0 = 1,
  max_iter = 1000,
  tol = 1e-8,
  seed = 42) {
  set.seed(seed)

  N <- length(x)
  alpha0_vec <- rep(alpha0, K)

  # Random uniform initialization for  $q(z_i = k)$ , row-normalized.

```

```

resp <- matrix(runif(N * K, min = 0, max = 1), nrow = N, ncol = K)
resp <- resp / rowSums(resp)

alpha <- alpha0_vec + colSums(resp)
a <- a0 + r * colSums(resp)
b <- b0 + colSums(resp * x)
converged <- FALSE

for (iter in seq_len(max_iter)) {
  resp_old <- resp

  # Update q(z_i): use log-space for stability.
  elog_pi <- digamma(alpha) - digamma(sum(alpha))
  elog_p <- digamma(a) - digamma(a + b)
  elog_1mp <- digamma(b) - digamma(a + b)

  for (i in seq_len(N)) {
    log_rho <- elog_pi + r * elog_p + x[i] * elog_1mp
    log_rho <- log_rho - max(log_rho)
    rho <- exp(log_rho)
    resp[i, ] <- rho / sum(rho)
  }

  # Update q(pi).
  alpha <- alpha0_vec + colSums(resp)

  # Update q(p_k).
  a <- a0 + r * colSums(resp)
  b <- b0 + colSums(resp * x)

  # Convergence check on responsibilities.
  delta <- max(abs(resp - resp_old))
  if (delta < tol) {
    converged <- TRUE
    break
  }
}

# Posterior means as point estimates.
pi_hat <- alpha / sum(alpha)
p_hat <- a / (a + b)
lambda_hat <- r * (1 - p_hat) / p_hat

```

```

list(
  r = r,
  iterations = iter,
  converged = converged,
  alpha = alpha,
  a = a,
  b = b,
  resp = resp,
  pi_hat = pi_hat,
  p_hat = p_hat,
  lambda_hat = lambda_hat
)
}

# Run for r in {10, 100, 1000}
# with vague priors a0=b0=alpha0=1
x <- load_vi_counts()
r_values <- c(10, 100, 1000)

vi_results <- lapply(
  r_values,
  function(r) {
    run_nb_vi(
      x = x,
      K = 2,
      r = r,
      a0 = 1,
      b0 = 1,
      alpha0 = 1,
      max_iter = 1000,
      tol = 1e-8,
      seed = 42
    )
  }
)

# Report p_k, pi_k, lambda_k
summary_table <- do.call(
  rbind,
  lapply(vi_results, function(res) {
    data.frame(
      r = res$r,

```

```

        component = seq_along(res$p_hat),
        p_k = res$p_hat,
        lambda_k = res$lambda_hat,
        pi_k = res$pi_hat,
        iterations = res$iterations,
        converged = res$converged
    )
  })
)

summary_table

```

	r	component	p_k	lambda_k	pi_k	iterations	converged
1	10	1	0.8954464	1.1676146	0.5869860	1000	FALSE
2	10	2	0.9359051	0.6848438	0.4130140	1000	FALSE
3	100	1	0.9876388	1.2515872	0.6349149	1000	FALSE
4	100	2	0.9952681	0.4754349	0.3650851	1000	FALSE
5	1000	1	0.9987306	1.2709925	0.6179106	1000	FALSE
6	1000	2	0.9995216	0.4785992	0.3820894	1000	FALSE